

Ouvidoria Inteligente: Triagem Semântica de Manifestações

UNIPE — Centro Universitário de João Pessoa · Processamento de Linguagem Natural / Engenharia de IA
Prof. Me. Ricardo Roberto de Lima · Equipe: [Integrante 1] · [Integrante 2] · [Integrante 3]

1. Contexto e decisões gerais

A ouvidoria recebe cerca de 4.000 manifestações por mês, e a triagem por palavra-chave não detecta duplicatas, não agrupa temas relacionados e perde contexto em textos longos. O protótipo usa representações vetoriais para atacar os três problemas.

- **Base de dados:** o arquivo *manifestacoes.json* citado no enunciado não veio com o material. Montamos uma base de 40 manifestações que segue a especificação: M001–M040, 5 categorias, 50 a 800 caracteres, 6 pares de duplicatas (15%) e 5 textos longos. Os textos de M003, M017, M008, M022 e M031 seguem o enunciado. Tudo lê *data/manifestacoes.json*, então basta trocar o arquivo pela base oficial.
- **Modelo principal:** *paraphrase-multilingual-MiniLM-L12-v2* (384D), comparado com o *all-MiniLM-L6-v2*. Os embeddings são normalizados, então produto escalar = cosseno.
- **Engenharia:** cache do modelo com *st.cache_resource* e dos embeddings com *st.cache_data*. Os notebooks são executados de ponta a ponta e entregues com as saídas.

2. Entrega 1 — Representações (analise_comparativa.ipynb)

Par	BoW	TF-IDF	Emb. multilíngue	Emb. inglês
M003 × M017 (buraco × esburacado)	0,069	0,051	0,626	0,542
M008 × M022 (sem médico × PSF)	0,000	0,000	0,674	0,554
M008 × M031 (sem médico × lâmpada)	0,000	0,000	0,125	0,480

M008 e M022 não compartilham **nenhuma** palavra ("médico" × "doutor", "posto" × "PSF"): para BoW e TF-IDF valem o mesmo que um par sem relação. O embedding multilíngue separa os casos com margem de 0,5 e, no corpus inteiro, dá similaridade média de 0,42 dentro da mesma categoria contra 0,22 entre categorias. O modelo em inglês dá 0,48 para posto de saúde × lâmpada: **o idioma do modelo importa**.

3. Entrega 2 — Duplicatas (deteccao_duplicatas.ipynb)

detectar_duplicatas(textos, limiar=0.85) devolve os pares com cosseno acima do limiar. Com 0,85 ela não acha nada: a duplicata mais parecida tem 0,818. Fizemos a varredura de 0,50 a 0,95 contra o gabarito:

Limiar	VP	FP	FN	Precisão	Recall	F1
0,60	6	10	0	0,375	1,000	0,545
0,65	4	2	2	0,667	0,667	0,667
0,70	2	0	4	1,000	0,333	0,500
0,85	0	0	6	—	0,000	0,000

Limiar escolhido: 0,65, que tem o melhor F1 e coincide com o percentil 99 da distribuição (0,643). O percentil funciona como limiar dinâmico em produção, onde não há gabarito. **Falsos positivos:** M016 × M033 (ponte no riacho × entulho no rio: textos longos com vocabulário de cenário parecido) e M014 × M020 (lixo × esgoto: mesmo tema, problemas diferentes). **Falsos negativos:** M003 × M017 (o modelo não sabe que "avenida principal" é a Av. Brasil) e M019 × M038 ("ponto" × "parada", "assaltos" × "é roubado"). Como fundir manifestações diferentes é pior que deixar passar uma duplicata, o sistema **sugere** os pares para um atendente confirmar.

4. Entrega 3 — Chunking (chunking_manifestacoes.ipynb)

Config	Chunks	Tam. médio	Coesão vizinhos	Cos. com original
A (150, 0)	32	93	0,225	0,525
B (150, 50)	32	96	0,260	0,537
C (300, 60)	14	214	0,441	0,726

- **Overlap:** subiu a coesão de 0,225 para 0,260, com todo o ganho concentrado onde uma frase foi cortada no meio (em A, M016 gerou o chunk "proteção lateral."). Quando a quebra cai numa fronteira de frase, o RecursiveCharacterTextSplitter não tem o que sobrepor, e A e B ficam idênticos.
- **Melhor configuração:** C (300, 60) preserva a denúncia (problema + local + pedido no mesmo chunk). B é melhor para achar detalhes: "peixes morrendo..." tem score 0,86 em B, 0,74 em C e só 0,42 com o texto inteiro.
- **Espaço 2D:** chunks da mesma manifestação têm similaridade ~2× maior que os de manifestações diferentes. M016 e M033 se misturam no t-SNE (o mesmo par do falso positivo), e os chunks se agrupam pelo assunto de cada trecho.

5. Entrega 4 — App (app_ouvidoria.py)

- **Busca Semântica:** tabela do Top-K (padrão 5) com faixa de cor (verde > 0,7, amarelo > 0,5, vermelho abaixo) e barra de similaridade, aviso de possível duplicata acima de 0,65 e categoria sugerida por voto ponderado dos vizinhos.
- **Base Completa:** tabela filtrável e botão que gera o heatmap de similaridade.
- **Espaço Vetorial:** PCA ou t-SNE colorido pela categoria oficial, mais o percentual de manifestações cujo vizinho mais próximo é da mesma categoria (70%). Saúde é a categoria mais coesa; infraestrutura se mistura com meio ambiente e iluminação pública com segurança, porque os clusters seguem o assunto do texto e não a categoria administrativa.
- **Chunking:** estratégia (Recursive ou Fixed-size), chunk_size e overlap ajustáveis, coesão entre vizinhos, similaridade de cada chunk com o texto inteiro, embeddings dos chunks e busca dentro do texto.
- **Duplicatas** (extra): limiar ajustável e pares lado a lado. Sidebar com modelo e Top-K.
- **Organização do código:** cada aba é uma função; a lógica compartilhada com os notebooks (carga da base, vetorização, detectar_duplicatas, chunking) fica em *nucleo_ouvidoria.py*.

6. Dificuldades e aprendizados

- O limiar de 0,85 sugerido não serve para este modelo: o limiar depende do modelo e deve ser calibrado com dados.
- Textos longos misturam assuntos num só vetor e geram falsos positivos, o que motiva o chunking.
- Categorias administrativas não coincidem com clusters semânticos. A classificação automática deve ser uma sugestão, com um humano no circuito.
- Próximos passos: busca híbrida com BM25 para nomes de ruas, geolocalização para duplicatas e um modelo maior (ex.: bge-m3).

Divisão de tarefas: [Integrante 1] — Entregas 1 e 2 · [Integrante 2] — Entregas 3 e 4 · Relatório em conjunto.